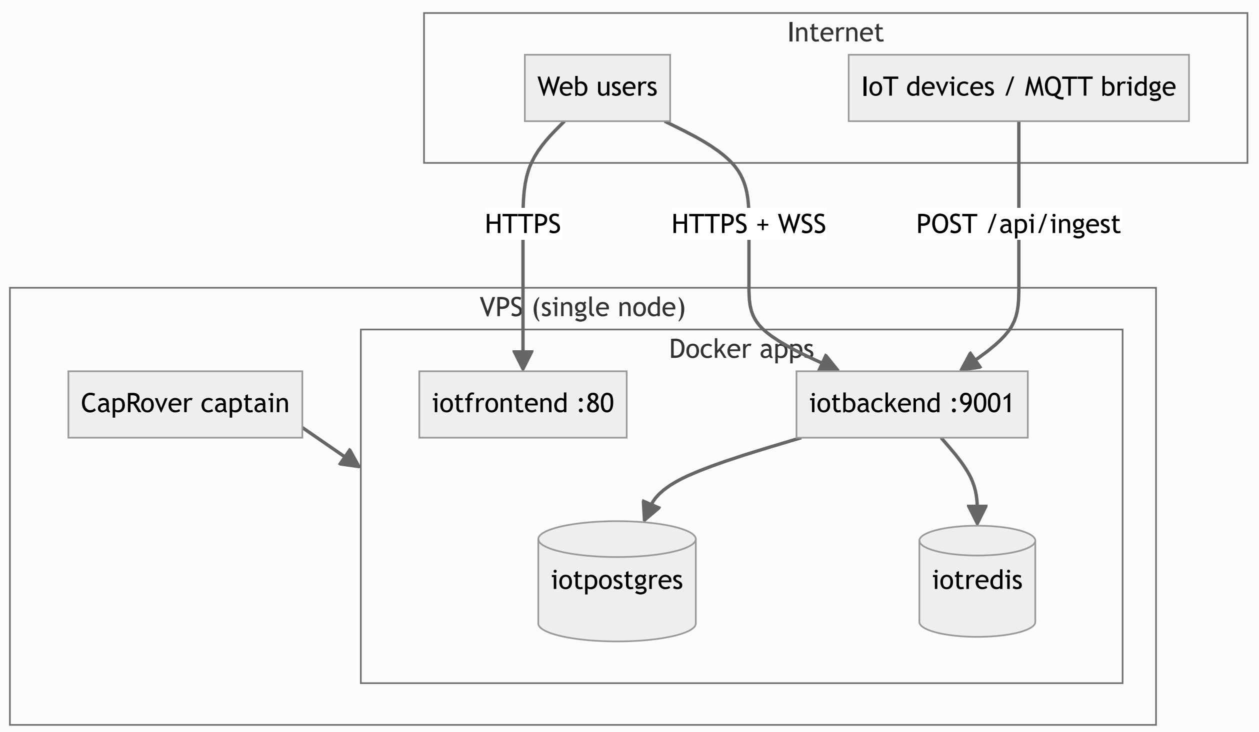
Smart AgriTech EMS Documentation

Deployment guide

Production deployment for Smart AgriTech EMS: **VPS + CapRover** with four apps (PostgreSQL, Redis, backend API, web dashboard). Optional **GitHub Actions** for continuous delivery.

> **Pricing note:** Figures below are approximate list prices (USD/EUR) as of mid-2026. Always confirm current pricing on each provider's website before purchasing.

Recommended topology



CapRover app	Image / source	Port	Public hostname
`iotpostgres`	One-Click PostgreSQL	5432 (internal)	No
`iotredis`	One-Click Redis	6379 (internal)	No
`iotbackend`	Repo `ems/ems-backend` Dockerfile	9001	`iotbackend.yourdomain.com`
`iotfrontend`	Repo `web_frontend` Dockerfile	80	`iotfrontend.yourdomain.com`

Reference env: `deploy/caprover.env.example`

Minimum server requirements

Workload	vCPU	RAM	Disk	Notes
Dev / demo (< 50 devices)	2	4 GB	40 GB SSD	All-in-one CapRover
Small production (50–500 devices)	4	8 GB	80 GB SSD	Enable Redis queue
Medium (500–2k devices)	4–8	16 GB	160 GB SSD	Tune `INGEST_WORKER_CONCURRENCY`
Large	8+	32 GB+	320 GB+	Split DB to managed Postgres

CapRover itself uses ~512 MB–1 GB RAM. PostgreSQL + Redis + Node backend dominate usage under ingest load.

Hosting options comparison

Budget VPS (self-managed CapRover)

Provider	Plan (example)	Specs	Price (approx.)	Traffic	Best for
Hetzner Cloud	CX23 / CPX12 (Gen3)	2 vCPU, 4 GB, 40 GB NVMe	€4–6/mo	20 TB (EU)	Best EU price/performance
Hetzner Cloud	CX33 / CPX22	4 vCPU, 8 GB, 80 GB	€8–12/mo	20 TB	Small production
DigitalOcean	Basic Droplet	2 vCPU, 4 GB, 80 GB	$24/mo	4 TB	Simple UI, global regions
DigitalOcean	Basic Droplet	4 vCPU, 8 GB, 160 GB	$48/mo	5 TB	Medium workloads
Linode (Akamai)	Shared 4 GB	2 vCPU, 4 GB, 80 GB	$24/mo	4 TB	US/APAC presence
Vultr	Cloud Compute	2 vCPU, 4 GB, 80 GB	$20/mo	3 TB	Many locations
OVH	VPS Starter	4 vCPU, 8 GB, 75 GB	~€6–10/mo	varies	EU budget option
Contabo	Cloud VPS S	4 vCPU, 8 GB, 200 GB	~€5–7/mo	32 TB	High disk, variable CPU

Free tier / always-free

Provider	Offer	Specs	Limits	EMS fit
Oracle Cloud	Always Free ARM	4 OCPU, 24 GB RAM (flex)	200 GB block storage, account approval	Excellent if approved — run full stack
Oracle Cloud	Always Free AMD	2× VM.Standard.E2.1.Micro	1 GB RAM each	Too small for Postgres+Redis+API together
Google Cloud	Free trial	$300 credit / 90 days	Trial only	Good for PoC
AWS	Free tier	t2/t3.micro 12 months	1 GB RAM	PoC only
Azure	Free account	$200 credit / 30 days	Trial	PoC only
Fly.io	Free allowance	Shared VMs	Sleep on idle, egress limits	Possible for API only; DB separate
Render	Free web services	512 MB	Spins down	Not recommended for production EMS

Recommendation: For zero cost long-term, **Oracle Always Free ARM** (24 GB) is the strongest option if your account is approved. For paid production, **Hetzner CX/CPX** offers the best value in EU.

Platform-as-a-Service (no CapRover)

Platform	Backend	Database	Est. monthly	Trade-offs
Railway	Node container	Postgres plugin	$5–20+ usage	Easy deploy; costs scale with usage
Render	Web service	Postgres	$25+	Managed; less control over ingest tuning
Fly.io	Machines	Fly Postgres	$10–40+	Global edge; more setup
DigitalOcean App Platform	Component	Managed DB	$30–60+	No CapRover; higher cost
Supabase	—	Postgres only	Free–$25	Use with separate API host
Neon	—	Serverless Postgres	Free–$19	Pair with VPS for API

EMS is optimized for **single VPS + CapRover** because ingest, Redis queues, WebSockets, and Postgres benefit from low latency on one network.

Managed databases (optional split)

Service	Entry price	Use when
DigitalOcean Managed Postgres	from ~$15/mo	Offload DB from VPS
AWS RDS	from ~$15–30/mo	Enterprise compliance
Supabase Pro	~$25/mo	Postgres + auth (auth unused if keeping EMS JWT)
Upstash Redis	Free tier / pay-per-request	Serverless Redis if VPS has no RAM for Redis

---

CapRover installation

1. Prepare VPS

```
# Ubuntu 22.04/24.04 recommended
sudo apt update && sudo apt upgrade -y
# Install Docker (official docs) — Docker 24+ or 29+ with matching CapRover tag
```

2. Run CapRover

Use **caprover/caprover:1.14.1+** if Docker API ≥ 1.44:

```
docker run -p 80:80 -p 443:443 -p 3000:3000 \
  -e ACCEPTED_TERMS=true \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v caprover-data:/captain \
  --name caprover --restart unless-stopped -d caprover/caprover:1.14.1
```

Port	Purpose
80 / 443	HTTP/S for apps
3000	CapRover admin UI (lock down firewall in production)

3. Initial CapRover setup

Open :3000 → Set root domain → Enable HTTPS Let's Encrypt → Create apps → Deploy one-click DBs → Deploy backend + frontend

1. Open `http://SERVER_IP:3000`

2. Set password; configure **root domain** (e.g. `yourdomain.com`)
3. Enable HTTPS (Let's Encrypt)
4. Create apps: `iotpostgres`, `iotredis`, `iotbackend`, `iotfrontend`

4. One-click databases

In CapRover → Apps → One-Click Apps:

- **PostgreSQL** → app name `iotpostgres` → note internal hostname `srv-captain--iotpostgres`
- **Redis** → app name `iotredis`

Build connection URLs for backend (CapRover internal DNS):

```
DATABASE_URL=postgresql://USER:PASS@srv-captain--iotpostgres:5432/postgres
REDIS_URL=redis://srv-captain--iotredis:6379
```

5. Backend app (`iotbackend`)

HTTP settings:

Setting	Value
Container HTTP Port	`9001`
Health check	`/health`
HTTPS	Enabled
Force HTTPS	Yes
WebSocket Support	**Yes**

Environment variables: see `deploy/caprover.env.example`

Critical vars:

```
NODE_ENV=production
PORT=9001
JWT_SECRET=<long random>
INGEST_API_KEY=<device ingest key>
CLIENT_URL=https://iotfrontend.yourdomain.com
DATABASE_URL=...
REDIS_URL=...
```

Deploy: upload `dist/caprover-backend.tgz` or use GitHub Actions.

On first start, `docker-entrypoint.sh` runs Prisma migrations and optional seed.

6. Frontend app (`iotfrontend`)

Setting	Value
Container HTTP Port	`80`
HTTPS	Enabled

API URL is **baked at build time** — no runtime env in container.

```
./scripts/caprover/pack-frontend.sh iotbackend.yourdomain.com
# Produces dist/caprover-frontend.tgz with:
#   VITE_API_URL=https://iotbackend.yourdomain.com/api
#   VITE_SOCKET_URL=https://iotbackend.yourdomain.com
```

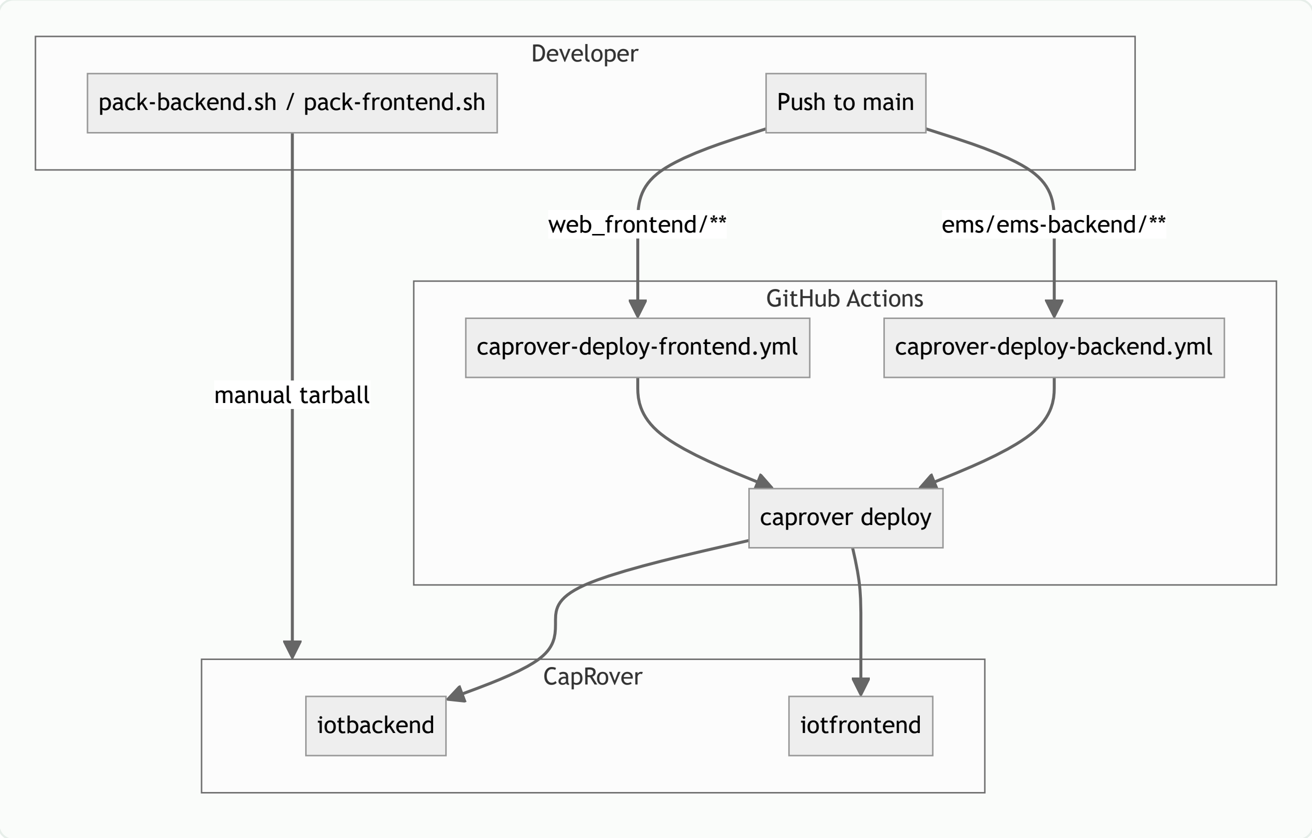
7. DNS records

Host	Type	Target
`captain.yourdomain.com`	A	VPS IP
`*.yourdomain.com`	A	VPS IP (wildcard for app subdomains)
or per-app	CNAME	captain domain

CapRover app hostnames:

- `iotbackend.yourdomain.com`
- `iotfrontend.yourdomain.com`

Deployment flow diagram



GitHub Actions secrets

Secret	Used by	Description
`CAPROVER_SERVER_URL`	Both	`https://captain.yourdomain.com`
`CAPROVER_BACKEND_APP`	Backend	`iotbackend`
`CAPROVER_BACKEND_TOKEN`	Backend	Per-app deploy token
`CAPROVER_FRONTEND_APP`	Frontend	`iotfrontend`
`CAPROVER_FRONTEND_TOKEN`	Frontend	Per-app deploy token
`CAPROVER_BACKEND_PUBLIC_HOST`	Frontend	`iotbackend.yourdomain.com` (no scheme)

Create tokens: CapRover → App → Deployment → Enable App Token.

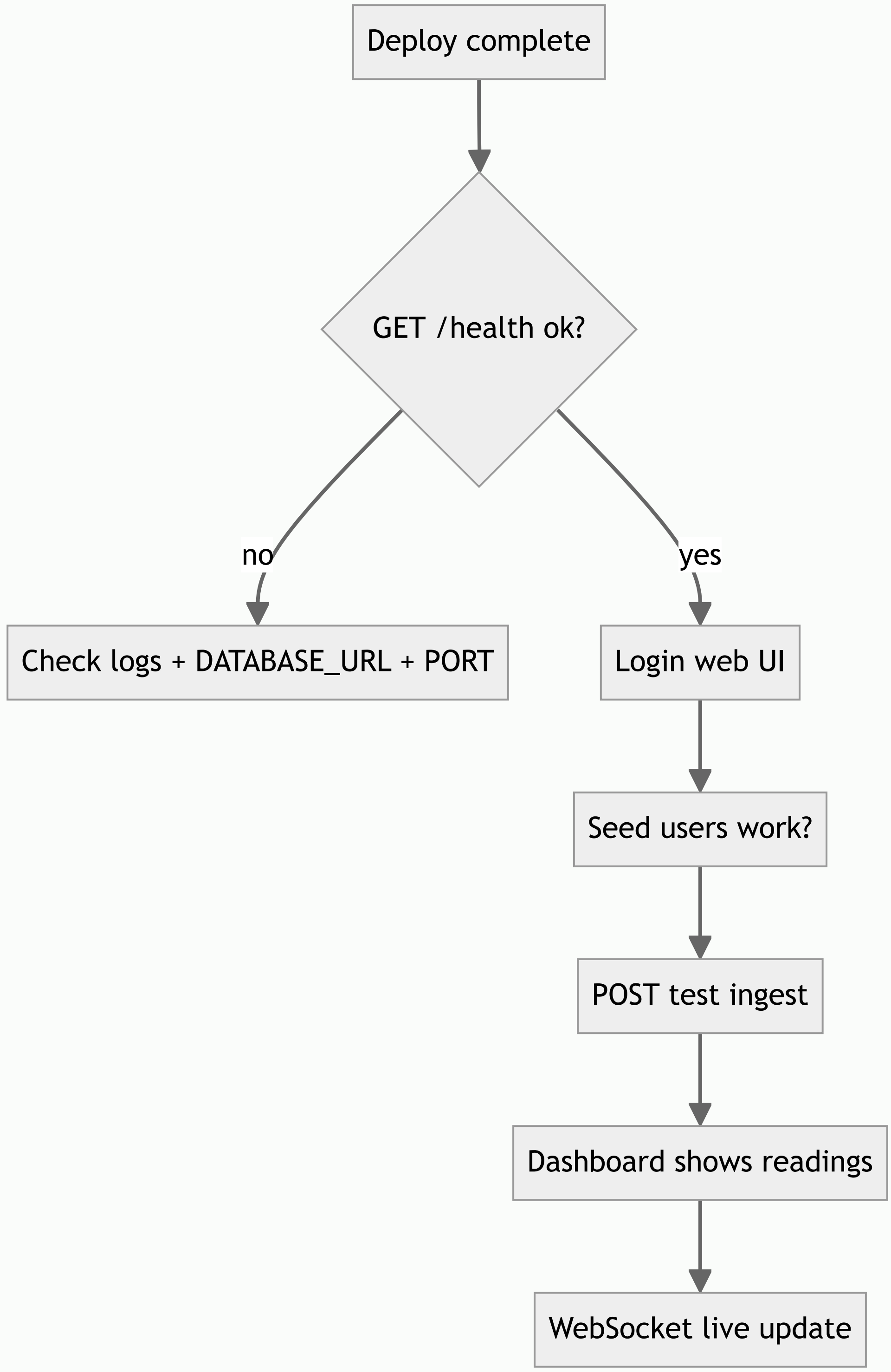
Do not use the CapRover admin password in CI.

Manual pack (Windows)

```
.\scripts\caprover\pack-all.ps1 -ApiHost "iotbackend.yourdomain.com" -WebHost "iotfrontend.y
```

Upload `dist/caprover-backend.tgz` and `dist/caprover-frontend.tgz` via CapRover UI.

Post-deploy checklist



Step	Command / action
Health	`curl https://iotbackend.yourdomain.com/health`
Login	`superadmin@ems.com` / `Admin@123456`
Ingest test	`curl -X POST .../api/ingest -H "x-api-key: KEY" -d '{...}'`
Migrations	Automatic on container start
Re-seed	`docker exec` into backend → `npm run seed` (skips if already seeded)
Logs	CapRover → App → App Logs

Optional: MQTT bridge

Devices publishing to MQTT can use the Python bridge at repo root:

```
# Dockerfile.mqtt-bridge or run on gateway hardware
python script.py   # forwards MQTT → HTTP POST /api/ingest
```

Deploy as a fifth CapRover app or on edge gateway (Raspberry Pi / industrial PC).

Security hardening

Item	Action
Firewall	Allow 80, 443 only; restrict 3000 to admin IP
Secrets	Rotate `JWT_SECRET`, `INGEST_API_KEY` in production
CORS	`CLIENT_URL` must match frontend origin exactly
DB	Never expose Postgres/Redis ports publicly
HTTPS	Force HTTPS on all CapRover apps
Backups	Daily `pg_dump` from `iotpostgres` volume

Scaling paths

Single VPS → Managed Postgres → Dedicated ingest workers → Read replica DATABASE_READ_URL

Stage	Change
1	Increase VPS size (RAM for Redis + Postgres)
2	Move Postgres to managed service
3	Add `INGEST_WORKER_CONCURRENCY` , tune batch settings
4	Set `DATABASE_READ_URL` for read-heavy dashboards
5	Multiple backend instances behind load balancer (advanced; requires shared Redis + sticky WS)

Cost examples (monthly estimates)

Scenario A — Demo / pilot

Item	Cost
Hetzner CX23 (4 GB)	~€5
Domain (.com)	~€1 amortized
Total	**~€6/mo**

Scenario B — Small production (200 devices, 1 msg/min)

Item	Cost
Hetzner CPX22 (8 GB)	~€10
Domain + backups storage	~€2
Total	**~€12/mo**

Scenario C — Managed split

Item	Cost
DO Droplet 4 GB (API + CapRover)	$24
DO Managed Postgres 1 GB	$15

Item	Cost
Total	**~$39/mo**

Scenario D — Free (Oracle ARM)

Item	Cost
OCI Always Free VM (24 GB ARM)	$0
Domain	~$12/year
Total	**~$1/mo** (domain only)

Troubleshooting

Symptom	Likely cause	Fix
502 on backend	Wrong container port	Set HTTP port `9001`
CORS errors	`CLIENT_URL` mismatch	Match exact frontend URL
WebSocket fails	WS disabled	Enable WebSocket Support on backend app
Empty dashboard	Wrong API URL in frontend	Rebuild frontend with correct `CAPROVER_BACKEND_PUBLIC_HOST`
Migrations fail	Bad `DATABASE_URL`	Use internal CapRover hostname
CapRover 3000 in use	Another Node process	`ss -tlnp | grep 3000`, stop conflicting service
`docker compose` not found	CapRover uses its own orchestration	Deploy via CapRover UI/CLI, not host compose

---

Local development vs production

Aspect	Local	Production
API	`localhost:5000`	`https://iotbackend.../api`
Frontend	Vite `:5173` + proxy	nginx static on CapRover
Database	Local Postgres	CapRover one-click
Redis	Optional local	Required for queued ingest
TLS	None	Let's Encrypt via CapRover

Related documents

- Architecture
- Backend
- Web frontend
- System flows
- Env reference: `deploy/caprover.env.example`